NoSQL y nuevas formas de almacenamiento — Pedidos de Food Store en MongoDB

TP Unidad 7 · Bases de Datos II · Tecnicatura Universitaria en Programación · UTN — **Ignacio Martín Rodríguez** — 29/10/2026 · PostgreSQL 16 · MongoDB (base **foodstore_nosql**) · repositorio **github.com/Nacho30/UTN-2026-BASE-DE-DATOS**, rama **tp-unidad-7**, carpeta **food-store-tp-u7/**

1. Subconjunto elegido y modelo documental (consigna 1)

Se toma el subconjunto **pedidos y detalle de pedidos** del esquema real (**schema.sql**). Intervienen cuatro tablas: **pedido** (PK **id_pedido**; FK **id_cliente** → cliente, ON DELETE RESTRICT), **pedido_producto** (PK compuesta **(id_pedido, id_producto)**; FK a pedido con CASCADE y a producto con RESTRICT; **precio_unitario** histórico, R4), y **cliente** y **producto**, de las que sólo se copian los datos que el pedido muestra.

Hallazgo: el Food Store real no tiene estado de pedido. Como la consigna lo pide, **sql/01_estado_y_pedidos_prueba.sql** agrega el ENUM **estado_pedido** (PENDIENTE, EN_PREPARACION, ENTREGADO, CANCELADO) y la columna **pedido.estado** (NOT NULL, DEFAULT PENDIENTE), con las convenciones del proyecto y sin tocar **schema.sql**.

Colección **pedidos** — un documento por pedido:

```
{ _id:        <id_pedido>              // identificador de origen conservado
  fecha:      Date (UTC)               // pedido.fecha
  estado:     "ENTREGADO" | ...        // pedido.estado (ENUM)
  forma_pago: "TARJETA" | ...          // pedido.forma_pago (ENUM)
  cliente:  { id_cliente, nombre, apellido }        // objeto embebido (copia)
  lineas: [ { id_producto, nombre, cantidad: Int32, // arreglo embebido = pedido_producto
              precio_unitario: Decimal128 } ],      // precio histórico, no el de lista
  total:      Decimal128 }             // Σ cantidad × precio_unitario (derivado)
```

2. Migración (consigna 2)

Se migran **6 pedidos con 11 líneas**. Los pedidos 1 a 4 son los de **datos_iniciales.sql**; los **5 y 6 son pedidos de prueba** de esta unidad (había sólo 4), insertados en PostgreSQL de modo que pasan por el trigger de RN1/RN2 y descuentan stock. Los documentos no se escribieron a mano: **sql/02_exportar_pedidos_json.sql** los arma desde PostgreSQL y **scripts/generar_insertmany.py** los vuelca en el **insertMany()** de **consultas_mongodb.js** (montos como **NumberDecimal**, cantidades como **NumberInt**, fechas como **ISODate**). La colección se crea con un validador **$jsonSchema** e índices sobre **estado** y **lineas.id_producto**.

_id	Fecha (-03)	Estado	Cliente	Líneas (producto × cant. a precio histórico)	Total
1	20/09 20:15	ENTREGADO	1 Ana Pérez	Hamburguesa clásica 2 × 8.000 · Gaseosa 500 ml 2 × 2.500	21.000
2	21/09 13:02	ENTREGADO	2 Bruno Díaz	Pizza muzzarella 1 × 12.000	12.000
3	22/09 21:40	ENTREGADO	1 Ana Pérez	Hamburguesa doble 1 × 11.000 · Agua mineral 1 × 1.800	12.800
4	23/09 12:10	EN_PREPARACION	3 Carla Gómez	Flan casero 2 × 3.500	7.000
5 (prueba)	24/09 20:30	PENDIENTE	2 Bruno Díaz	Hamburguesa doble 2 × 11.000 · Gaseosa 500 ml 2 × 2.500	27.000
6 (prueba)	25/09 13:45	ENTREGADO	3 Carla Gómez	Pizza muzzarella 1 × 12.000 · Agua mineral 1 × 1.800 · Flan 1 × 3.500	17.300

3. Reglas de integridad y qué se embebió (consigna 3)

Regla 1 — Precio histórico (R4). Cada línea lleva el **precio_unitario** de **pedido_producto**, nunca **producto.precio**. No es teórico: el pedido 1 vendió la hamburguesa clásica a 8.000 y hoy su precio de lista es 8.500; migrar con el precio actual daría un total de 22.000 en vez de 21.000 (**03_contraste_ia_postgresql.txt**).

Regla 2 — Cantidades y montos válidos y total coherente. Se conservan los CHECK **cantidad > 0** y **precio_unitario ≥ 0**, y los dominios de estado y forma de pago, como validador **$jsonSchema** (con **validationAction: "error"** y prueba de rechazo de una línea con cantidad 0). Como **total** es un dato derivado que PostgreSQL no guarda (3FN), el script verifica que en todos los documentos sea igual a Σ cantidad × precio_unitario.

Por qué embebido. Las líneas no existen sin su pedido (en PostgreSQL se borran en CASCADE), son pocas por pedido, se leen siempre junto con él y se escriben una sola vez: es la relación «contiene» para la que la documentación de MongoDB recomienda embeber, y deja el pedido completo en una lectura y una escritura atómica. Del cliente y del producto se copian sólo id y nombre (lo que el pedido muestra), conservando sus id para volver a PostgreSQL. **Precio de lista, stock, activo, email y teléfono quedan fuera**: cambian independientemente del pedido y copiarlos duplicaría datos que no son del momento de la venta.

4. Consultas en MongoDB (consigna 4)

	Instrucción en consultas_mongodb.js	Resultado
a	find({}, {_id: 1, estado: 1}).sort({_id: 1})	6 pedidos: 1, 2, 3 y 6 ENTREGADO; 4 EN_PREPARACION; 5 PENDIENTE
b	find({estado: "ENTREGADO"}, {_id: 1, estado: 1, fecha: 1})	4 pedidos: 1, 2, 3, 6
c	find({"lineas.id_producto": 4}, {_id: 1, estado: 1, lineas: {$elemMatch: {id_producto: 4}}}) — producto 4 = Gaseosa 500 ml; se filtra por id, no por el nombre copiado	2 pedidos: 1, 5
d	find({total: {$gt: NumberDecimal("16000")}}, {_id: 1, estado: 1, total: 1})	3 pedidos: 1 (21.000), 5 (27.000), 6 (17.300)

Umbral de d). 16.000 es el ticket promedio de los pedidos migrados (16.183,33, calculado con **$avg** en el script y con **AVG** en PostgreSQL) redondeado: la consulta responde «qué pedidos están por encima del ticket promedio». Un umbral fijo como 10.000 dejaría afuera sólo al pedido 4 y no distinguiría nada.

5. Verificación contra PostgreSQL (consigna 5)

En **consultas_postgresql.sql**: b) **WHERE estado = 'ENTREGADO'**; c) **EXISTS** sobre **pedido_producto** con **id_producto = 4** (la PK compuesta impide repetir el producto en un pedido, así que un JOIN daría lo mismo, pero EXISTS no duplicaría pedidos si eso cambiara). Salida completa en **02_consultas_postgresql.txt**.

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	IDs PostgreSQL	IDs MongoDB	Coinciden los IDs
Estado seleccionado: ENTREGADO	4	4	1, 2, 3, 6	1, 2, 3, 6	Sí
Producto seleccionado: 4 Gaseosa 500 ml	2	2	1, 5	1, 5	Sí

Diferencias detectadas entre los modelos (no cambian b ni c, pero sí cambiarían otras consultas y por eso se corrigieron o documentaron):

Diferencia	Efecto	Corrección
Fecha: PostgreSQL muestra la hora local (-03); MongoDB guarda Date en UTC.	El pedido 3 (22/09 21:40 local) es del 23/09 en UTC: un filtro «pedidos del 22/09» en Mongo lo perdería.	Filtrar por rango con el desfase: 22/09 03:00Z a 23/09 03:00Z.
Precio: tomar producto.precio en lugar de pedido_producto.precio_unitario.	Pedido 1: total 22.000 en vez de 21.000; d) y cualquier suma de facturación no coincidirían.	El export usa precio_unitario; el script verifica total = Σ líneas.
Nombre del producto: en Mongo es una copia.	Si el catálogo renombra un producto, buscar por nombre dejaría de coincidir con PostgreSQL.	La consulta c) filtra por lineas.id_producto, el id conservado.
Tipos numéricos: en mongosh un número sin tipo es double.	Montos double pueden arrastrar error binario (0,1 + 0,2 ≠ 0,3) frente a NUMERIC(10,2).	NumberDecimal para montos y NumberInt para cantidades; el validador lo exige.

6. Consulta a la IA (consigna 6)

Herramienta: Claude (Anthropic), 29/10/2026. El prompt describió el modelo actual (4 tablas, 3FN, PK compuesta, **precio_unitario** histórico, CHECK y trigger de stock), el acceso frecuente al pedido completo y la necesidad de conservar la coherencia de cantidades y precios. Prompt y respuesta resumida en **docs/ia/consulta_ia.md**. La IA recomendó mantener PostgreSQL como sistema de registro y usar MongoDB, si hiciera falta, como réplica de lectura.

7. Contraste de afirmaciones (consigna 7)

Afirmación de la IA	Contraste	Veredicto
«En PostgreSQL leer un pedido completo exige unir pedido, cliente, pedido_producto y producto; en MongoDB, con las líneas embebidas, es una sola lectura.»	**Laboratorio:** el EXPLAIN del pedido 1 muestra 3 joins sobre 4 tablas, todos por PK o índice (**pedido_pkey**, **cliente_pkey**, **pedido_producto_pkey**). En MongoDB, **findOne({_id: 1})** devuelve el documento con cliente y líneas, sin **$lookup**. **Documentación** (Embedded Data): embeber permite obtener datos relacionados en una sola operación.	**Correcta**, con matiz: los joins son por clave e indexados, así que «4 tablas» no implica lentitud con este volumen.
«Las restricciones de cantidad y precio se pueden expresar con $jsonSchema, de forma parecida a un CHECK.»	**Documentación** (Specify JSON Schema Validation): **bsonType**, **minimum**, **enum** y **minItems** están soportados y con **validationAction: "error"** se rechaza la escritura. Pero el validador sólo evalúa el documento que se escribe: no puede consultar otra colección. **Laboratorio:** RN1 (producto activo) y RN2 (stock: los pedidos de prueba bajaron, p. ej., la hamburguesa doble de 9 a 7) dependen de la fila de **producto**.	**Parcialmente correcta**: cubre la regla 2 de la consigna 3, no RN1/RN2 ni las FK, que en MongoDB pasarían a la aplicación o a una transacción multidocumento.

También se verificó la afirmación de atomicidad: según **Atomicity and Transactions**, las escrituras son atómicas a nivel de documento único y las transacciones distribuidas cuestan más y no reemplazan un buen diseño de esquema; un pedido con sus líneas embebidas se inserta entero o no se inserta.

Conclusión

Conviene **mantener PostgreSQL** como almacenamiento de los pedidos de Food Store. **Criterio 1, integridad:** las reglas que más importan al pedido cruzan entidades: el trigger valida que el producto esté activo y descuenta stock con la fila bloqueada, y las FK impiden líneas huérfanas. El validador de MongoDB sólo ve el documento, así que esas reglas pasarían a la aplicación o a transacciones multidocumento, que la propia documentación desaconseja como sustituto del diseño. **Criterio 2, patrón de acceso y rendimiento:** el documento embebido coincide con la lectura frecuente del pedido completo y lo escribe atómicamente; es la ventaja real de MongoDB. Pero en PostgreSQL esa lectura son joins por clave primaria e índice, sin costo relevante con el volumen del proyecto. **Criterio 3, coherencia de montos:** la migración exige cuidar tipos (Decimal128), precio histórico y zona horaria; cada copia es una fuente posible de diferencias. Combinar tecnologías tendría sentido sólo si se midiera un cuello de botella en la lectura de pedidos: MongoDB como réplica de lectura alimentada desde PostgreSQL, nunca como fuente de verdad.

Anexo · Evidencias de ejecución

PostgreSQL — consultas b) y c) (**docs/evidencia/02_consultas_postgresql.txt**):

```
== b) Pedidos ENTREGADO
 id_pedido |  estado   |         fecha
-----------+-----------+------------------------
         1 | ENTREGADO | 2026-09-20 20:15:00-03
         2 | ENTREGADO | 2026-09-21 13:02:00-03
         3 | ENTREGADO | 2026-09-22 21:40:00-03
         6 | ENTREGADO | 2026-09-25 13:45:00-03
(4 rows)
 cantidad |    ids
----------+------------
        4 | 1, 2, 3, 6
(1 row)
== c) Pedidos con el producto 4
 id_pedido |  estado   | cantidad | precio_unitario
-----------+-----------+----------+-----------------
         1 | ENTREGADO |        2 |         2500.00
         5 | PENDIENTE |        2 |         2500.00
(2 rows)
 cantidad | ids
----------+------
        2 | 1, 5
(1 row)
```

PostgreSQL — umbral de d) y contraste de precio y fechas (**02_consultas_postgresql.txt**, **03_contraste_ia_postgresql.txt**):

```
== d) Ticket promedio y pedidos con total > 16000
 promedio
----------
 16183.33
(1 row)
 id_pedido |  estado   |  total
-----------+-----------+----------
         1 | ENTREGADO | 21000.00
         5 | PENDIENTE | 27000.00
         6 | ENTREGADO | 17300.00
(3 rows)

== Si se migrara con producto.precio en vez de precio_unitario
 id_pedido | total_correcto | total_con_precio_actual
-----------+----------------+-------------------------
         1 |       21000.00 |                22000.00
(1 row)
== Fechas: día local (-03) vs día UTC (como queda en MongoDB)
 id_pedido | dia_local  |  dia_utc
-----------+------------+------------
         3 | 2026-09-22 | 2026-09-23
(1 row)
```

MongoDB — salida de **consultas_mongodb.js** (**docs/evidencia/04_salida_mongodb.txt**):

```
== Documentos migrados: 6
== Pedidos con total inconsistente (esperado: ninguno)
  []
== Validador OK, rechazó cantidad 0: Document failed validation
== Pedido completo _id 1
  {_id: 1, fecha: ISODate('2026-09-20T23:15:00.000Z'), estado: 'ENTREGADO', forma_pago: 'TARJETA', cliente:
      {id_cliente: 1, nombre: 'Ana', apellido: 'Pérez'}, lineas: [{id_producto: 1, nombre: 'Hamburguesa clásica',
      cantidad: 2, precio_unitario: Decimal128('8000.00')}, {id_producto: 4, nombre: 'Gaseosa 500 ml', cantidad: 2,
      precio_unitario: Decimal128('2500.00')}], total: Decimal128('21000.00')}
== a) Todos los pedidos
  [{_id: 1, estado: 'ENTREGADO'},
  {_id: 2, estado: 'ENTREGADO'},
  {_id: 3, estado: 'ENTREGADO'},
  {_id: 4, estado: 'EN_PREPARACION'},
  {_id: 5, estado: 'PENDIENTE'},
  {_id: 6, estado: 'ENTREGADO'}]
== b) Pedidos ENTREGADO
  [{_id: 1, fecha: ISODate('2026-09-20T23:15:00.000Z'), estado: 'ENTREGADO'},
  {_id: 2, fecha: ISODate('2026-09-21T16:02:00.000Z'), estado: 'ENTREGADO'},
  {_id: 3, fecha: ISODate('2026-09-23T00:40:00.000Z'), estado: 'ENTREGADO'},
  {_id: 6, fecha: ISODate('2026-09-25T16:45:00.000Z'), estado: 'ENTREGADO'}]
  cantidad: 4
== c) Pedidos con el producto 4 (Gaseosa 500 ml)
  [{_id: 1, estado: 'ENTREGADO', lineas: [{id_producto: 4, nombre: 'Gaseosa 500 ml', cantidad: 2, precio_unitario:
      Decimal128('2500.00')}]},
  {_id: 5, estado: 'PENDIENTE', lineas: [{id_producto: 4, nombre: 'Gaseosa 500 ml', cantidad: 2, precio_unitario:
      Decimal128('2500.00')}]}]
  cantidad: 2
== d) Ticket promedio (justifica el umbral)
  [{_id: null, promedio: Decimal128('16183.333333333333333333333333333')}]
== d) Pedidos con total > 16000
  [{_id: 1, estado: 'ENTREGADO', total: Decimal128('21000.00')},
  {_id: 5, estado: 'PENDIENTE', total: Decimal128('27000.00')},
  {_id: 6, estado: 'ENTREGADO', total: Decimal128('17300.00')}]
```